

Detailed Academic Notes: System Modeling

UNIT-II: System Modeling, Architectural Design, and Design and Implementation

Chapter 5 – System Modeling

This chapter covers the following key topics:

- Context models
- Interaction models
- Structural models
- Behavioral models
- Model-driven engineering

System Modeling

- **Definition:** System modeling is the process of developing abstract models of a system, with each model presenting a distinct view or perspective of that system.
- **Graphical Notation:** System modeling commonly involves representing a system using graphical notation, which is almost always based on the Unified Modeling Language (UML) notations.
- **Purpose:** System modeling aids the analyst in understanding the functionality of the system. Models also serve as a communication tool with customers.

Existing and Planned System Models

- **Models of the Existing System:** These are utilized during requirements engineering to clarify the current system's functions and to discuss its strengths and weaknesses, which in turn inform requirements for a new system.
- **Models of the New System:** Used during requirements engineering to explain proposed requirements to other system stakeholders. Engineers employ these models for discussing design proposals and for documenting the system for implementation.
- **Model-Driven Engineering (MDE):** In an MDE process, it is possible to generate a complete or partial system implementation directly from the system model.

System Perspectives

System modeling encompasses various perspectives, each offering a unique view:

- **External Perspective:** This involves modeling the context or environment in which the system operates.
- **Interaction Perspective:** This focuses on modeling the interactions between a system and its environment, or among the components within a system.
- **Structural Perspective:** This deals with modeling the organization of a system or the structure of the data processed by the system.
- **Behavioral Perspective:** This involves modeling the dynamic behavior of the system and how it responds to various events.

UML Diagram Types

The Unified Modeling Language (UML) provides several types of diagrams for system modeling:

- **Activity diagrams:** Illustrate the activities involved in a process or in data processing.
- **Use case diagrams:** Show the interactions between a system and its environment.
- **Sequence diagrams:** Depict interactions between actors and the system, as well as between different system components.
- **Class diagrams:** Show the object classes within the system and the associations linking these classes.
- **State diagrams:** Demonstrate how the system reacts to internal and external events.

Use of Graphical Models

Graphical models can be used in several ways:

- **Facilitating Discussion:** They serve as a means to discuss an existing or proposed system. In this context, incomplete or incorrect models are acceptable as their primary role is to support dialogue.
- **Documenting an Existing System:** Models can accurately represent an existing system, although they do not necessarily need to be complete.
- **Detailed System Description for Implementation:** When used for generating a system implementation, models must be both correct and complete.

Context Models

- **Purpose:** Context models are used to illustrate the operational context of a system. They show what lies outside the defined system boundaries.
- **Influences:** Social and organizational concerns can influence the decision regarding the positioning of system boundaries.
- **Architectural Models:** These models specifically show the system and its relationships with other systems.

System Boundaries

- **Definition:** System boundaries are established to define precisely what is considered inside and what is outside the system being developed.
- **Dependencies:** They indicate other systems that are used by or depend on the system under development.
- **Impact on Requirements:** The placement of the system boundary significantly impacts the system requirements.
- **Political Judgment:** Defining a system boundary is often a political judgment. There can be pressures to set boundaries in ways that increase or decrease the influence or workload of different parts of an organization.

Process Perspective

- **Distinction from Context Models:** While context models show other systems in the environment, they do not detail how the system being developed is used within that environment.
- **Purpose of Process Models:** Process models reveal how the system under development is integrated and used within broader business processes.
- **UML Activity Diagrams:** These diagrams can be employed to define business process models.
 - *Example:* A "Process model of involuntary detention" is mentioned as an application, but its detailed content is not provided.

Interaction Models

- **Importance of Modeling User Interaction:** It is crucial for identifying user requirements.
- **Importance of Modeling System-to-System Interaction:** This highlights potential communication problems that may arise between systems.

- **Importance of Modeling Component Interaction:** This helps in understanding whether a proposed system structure is likely to achieve the required system performance and dependability.
- **UML Diagrams for Interaction Modeling:** Use case diagrams and sequence diagrams are commonly used for interaction modeling.

Use Case Modeling

- **Origin and Integration:** Use cases were originally developed to support requirements elicitation and are now incorporated into the UML.
- **Representation:** Each use case represents a discrete task that involves external interaction with a system.
- **Actors:** Actors in a use case can be people or other systems.
- **Forms of Representation:** Use cases are typically represented diagrammatically to provide an overview, and in a more detailed textual form.

Example: Transfer-data Use Case in the Mentcare System

This is a specific use case from the Mentcare system. Its detailed textual description is as follows:

MHC-PMS: Transfer data

- **Actors:** Medical receptionist, patient records system (PRS)
- **Description:** A receptionist can transfer data from the Mentcare system to a general patient record database maintained by a health authority. The information transferred can be either updated personal information (e.g., address, phone number) or a summary of the patient's diagnosis and treatment.
- **Data:** Patient's personal information, treatment summary
- **Stimulus:** User command issued by medical receptionist
- **Response:** Confirmation that PRS has been updated
- **Comments:** The receptionist must possess appropriate security permissions to access the patient information and the PRS.

(Further use cases in the Mentcare system involving the 'Medical Receptionist' role are mentioned but not detailed.)

Sequence Diagrams

- **Definition:** Sequence diagrams are a part of the UML. They are used to model the interactions between the actors and the objects within a system.

- **Purpose:** A sequence diagram illustrates the sequence of interactions that occur during a specific use case or a particular instance of a use case.
- **Structure:**
 - The objects and actors involved are listed along the top of the diagram.
 - A dotted line is drawn vertically from each object/actor.
 - Interactions between objects are shown by annotated arrows.

(Examples like "Sequence diagram for View patient information" and "Sequence diagram for Transfer Data" are mentioned, but the diagrams themselves and their step-by-step explanations are not provided in the source.)

Structural Models

- **Purpose:** Structural models of software display the organization of a system in terms of its constituent components and their relationships.
- **Types:**
 - **Static models:** Show the structure of the system design.
 - **Dynamic models:** Show the organization of the system as it is executing.
- **Application:** Structural models are created when discussing and designing the system architecture.

Class Diagrams

- **Usage:** Class diagrams are used in the development of an object-oriented system model. They show the classes within a system and the associations (relationships) between these classes.
- **Object Class:** An object class can be understood as a general definition for a type of system object.
- **Association:** An association represents a link between classes, indicating some form of relationship between them.
- **Real-world Representation:** In the early stages of the software engineering process, objects in these models represent real-world entities, such as a patient, a prescription, or a doctor.

(References to "UML classes and association," "Classes and associations in the MHC-PMS," and "The Consultation class" are made, but no specific diagrams or detailed explanations of these are provided in the source.)

Generalization

- **Concept:** Generalization is a common technique used to manage complexity. Instead of learning the detailed characteristics of every individual entity, entities are grouped into more general classes (e.g., animals, cars, houses). The characteristics of these general classes are then learned.
- **Inference:** This allows for the inference that different members of these general classes share common characteristics (e.g., squirrels and rats are rodents).
- **Use in System Modeling:** In system modeling, examining classes for potential generalization is often useful. If changes are proposed, this approach means one does not need to inspect every single class in the system to determine if it is affected.
- **Implementation:** In object-oriented languages like Java, generalization is implemented through class inheritance mechanisms.
- **Attributes and Operations:** In a generalization hierarchy, attributes and operations associated with higher-level classes are also associated with their lower-level classes. Lower-level classes (subclasses) inherit attributes and operations from their superclasses.

(Mentions of "A generalization hierarchy" and "A generalization hierarchy with added detail" are made, but no detailed diagrams or explanations are provided in the source.)

Object Class Aggregation Models

- **Purpose:** An aggregation model illustrates how classes that are collections are composed of other classes.
- **Relationship:** Aggregation models are similar to the "part-of" relationship found in semantic data models.

(Reference to "The aggregation association" is made, but no detailed diagrams or explanations are provided in the source.)

Behavioral Models

- **Definition:** Behavioral models are models of the dynamic behavior of a system as it executes. They depict what occurs or is intended to occur when a system responds to a stimulus from its environment.
- **Types of Stimuli:** Stimuli can generally be categorized into two types:
 - **Data:** Data arrives that needs to be processed by the system.
 - **Events:** An event happens that triggers system processing. Events may or may not have associated data.

Data-Driven Modeling

- **Application:** Many business systems are primarily data-processing systems, driven by data input, with relatively little external event processing.
- **Purpose:** Data-driven models show the sequence of actions involved in processing input data and generating a corresponding output.
- **Utility:** They are particularly useful during requirements analysis as they can illustrate end-to-end processing within a system.

(Examples like "An activity model of the insulin pump's operation" and "Order processing" are mentioned, but their detailed content or diagrams are not provided.)

Event-Driven Modeling

- **Application:** Real-time systems are frequently event-driven, characterized by minimal data processing. For example, a landline phone switching system responds to events such as 'receiver off hook' by producing a dial tone.
- **Purpose:** Event-driven modeling demonstrates how a system responds to both external and internal events.
- **Underlying Assumption:** It is based on the assumption that a system possesses a finite number of states, and events (stimuli) can cause a transition from one state to another.

State Machine Models

- **Purpose:** These models describe the behavior of a system in response to external and internal events. They are often used for modeling real-time systems because they show the system's responses to stimuli.
- **Representation:**
 - System states are represented as nodes.
 - Events are represented as arcs between these nodes.
 - When an event occurs, the system transitions from one state to another.
- **UML Integration:** Statecharts are an integral part of the UML and are used to represent state machine models.

Example: Microwave Oven Operation (States and Stimuli)

A state diagram for a microwave oven is mentioned, along with detailed descriptions of its states and stimuli.

States for the Microwave Oven

State	Description
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows 'Half power'.
Full power	The oven power is set to 600 watts. The display shows 'Full power'.
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'.
Enabled	Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'.

Stimuli for the Microwave Oven

Stimulus	Description
Half power	The user has pressed the half-power button.
Full power	The user has pressed the full-power button.
Timer	The user has pressed one of the timer buttons.
Number	The user has pressed a numeric key.
Door open	The oven door switch is not closed.
Door closed	The oven door switch is closed.
Start	The user has pressed the Start button.
Cancel	The user has pressed the Cancel button.

Model-Driven Engineering (MDE)

- **Definition:** MDE is an approach to software development where models, rather than traditional programs, serve as the primary outputs of the development process.
- **Automation:** The programs that execute on a hardware/software platform are then automatically generated from these models.
- **Advantage (Proponents' View):** Proponents argue that MDE elevates the level of abstraction in software engineering, freeing engineers from concerns about programming language specifics or execution platform details.

Usage of Model-Driven Engineering

- **Current Stage:** MDE is still in an early stage of development, and its significant impact on software engineering practice remains uncertain.
- **Pros:**
 - Allows systems to be considered at higher levels of abstraction.
 - Automatic code generation makes it cheaper to adapt systems to new platforms.
- **Cons:**
 - Models created for abstraction may not be suitable for direct implementation.

Model Driven Architecture (MDA)

- **Precursor:** MDA was the precursor to the more general concept of Model-Driven Engineering.
- **Approach:** MDA is a model-focused approach to software design and implementation that uses a subset of UML models to describe a system.
- **Abstraction Levels:** Models are created at different levels of abstraction.
- **Goal:** In principle, it aims to generate a working program without manual intervention, starting from a high-level, platform-independent model.

Types of Models in MDA

MDA specifies different types of models based on their level of abstraction:

- **Computation Independent Model (CIM):**
 - These models represent the important domain abstractions used within a system.
 - CIMs are sometimes referred to as domain models.
- **Platform Independent Model (PIM):**
 - These models describe the operation of the system without any reference to its specific implementation details.
 - The PIM is typically described using UML models that illustrate the static system structure and its responses to external and internal events.
- **Platform Specific Models (PSM):**
 - These are transformations of the platform-independent model.
 - A separate PSM is created for each target application platform.
 - In theory, there can be multiple layers of PSMs, with each layer adding more platform-specific details.

(References to "MDA transformations" and "Multiple platform-specific models" are made, but no detailed diagrams or explanations are provided in the source.)

Agile Methods and MDA

- **Claim vs. Reality:** Developers of MDA claim it supports an iterative development approach and can be used within agile methods. However, the concept of extensive up-front modeling inherent in MDA contradicts the fundamental principles of the agile manifesto.
- **Potential for Integration:** If model transformations can be fully automated and a complete program generated from a PIM, then, in principle, MDA could align with an agile development process, as no separate coding would be required.

Adoption of MDA

Several factors have limited the widespread adoption of MDE/MDA:

- **Tool Support:** Specialized tool support is necessary to convert models from one level to another.
- **Tool Availability:** There is limited availability of such tools, and organizations may need to adapt and customize existing tools to their specific environment.
- **Reliance on Small Companies:** For long-lifetime systems typically developed using MDA, companies are reluctant to develop their own tools or depend on small companies that might cease operations.
- **Abstraction vs. Implementation:** Abstractions that are useful for discussing software design may not be the most appropriate for direct implementation.
- **Focus on Complex Systems:** For most complex systems, implementation itself is often not the primary challenge; issues like requirements engineering, security, dependability, integration with legacy systems, and testing are often more significant.
- **Cost-Benefit for Specific Systems:** The arguments for platform-independence are mainly valid for large, long-lifetime systems. For software products and information systems, the savings from MDA's use are likely to be outweighed by the costs of its introduction and necessary tooling.
- **Rise of Agile Methods:** The widespread adoption of agile methods over the same period that MDA was evolving has diverted attention away from model-driven approaches.

UNIT-II: System Modeling and Architectural Design Notes

1. Introduction to System Modeling and Architectural Design

- **System Modeling** involves various aspects such as Context models, Interaction models, Structural models, Behavioural models, and Model-driven architecture ``.
- **Architectural Design** encompasses design decisions, architectural views, architectural patterns, and architectures ``.
- **Design and Implementation** focuses on Object-oriented design using UML, Design patterns, Implementation issues, and Open-source development ``.

2. Chapter 6 – Architectural Design

This chapter covers key topics including:

- Architectural design decisions ``
- Architectural views ``
- Architectural patterns ``
- Application architectures ``

3. Architectural Design

- **Definition:** Architectural design is primarily concerned with understanding how a software system should be organized and then designing its overall structure ``.
- **Significance:** It serves as the critical link between design and requirements engineering . It identifies the main structural components within a system and defines the relationships between them.
- **Output:** The architectural design process produces an architectural model, which describes how the system is organized as a collection of communicating components ``.

4. Agility and Architecture

- In agile processes, it is generally accepted that designing an overall system architecture is an early and important stage ``.
- Refactoring a system's architecture is typically expensive due to its widespread impact on many system components ``.
 - **Example:** The architecture of a packing robot control system would be an example where refactoring would be costly ``.

5. Architectural Abstraction

Architectural abstraction exists at two main levels:

- **Architecture in the Small:** This level focuses on the architecture of individual programs . It is concerned with how an individual program is decomposed into its constituent components.
- **Architecture in the Large:** This addresses the architecture of complex enterprise systems . These systems often include other systems, programs, and program components. Such enterprise systems are typically distributed across different computers, which may be owned and managed by various companies ``.

6. Advantages of Explicit Architecture

Having an explicit architecture provides several benefits:

- **Stakeholder Communication:** The architecture can serve as a central point of discussion for system stakeholders ``.
- **System Analysis:** An explicit architecture enables the analysis of whether the system can meet its non-functional requirements ``.
- **Large-Scale Reuse:**
 - The defined architecture itself may be reusable across a range of different systems ``.
 - This also facilitates the development of product-line architectures ``.

7. Architectural Representations

- **Common Method:** Simple, informal block diagrams depicting entities and relationships are the most frequently used method for documenting software architectures ``.
- **Criticisms of Block Diagrams:**
 - They often lack semantics, meaning they do not convey enough meaning about the components ``.
 - They typically do not show the specific types of relationships between entities ``.
 - They may not illustrate the visible properties of entities within the architecture ``.
- **Dependency on Models:** Effective architectural representation depends on the use of architectural models . The specific requirements for model semantics are determined by how these models are intended to be used.

7.1 Box and Line Diagrams

- **Characteristics:** These diagrams are highly abstract . They do not explicitly show the nature of component relationships or the externally visible properties of the sub-systems.

- **Usefulness:** Despite their abstract nature, they are valuable for communication with stakeholders and for project planning ``.

8. Use of Architectural Models

Architectural models serve two primary purposes:

- **Facilitating Discussion about System Design:**
 - A high-level architectural view, free from excessive detail, is beneficial for communication with system stakeholders and for project planning ``.
 - Stakeholders can easily relate to and understand an abstract view of the system, enabling them to discuss the system as a whole without being overwhelmed by minor details ``.
- **Documenting a Designed Architecture:**
 - The goal here is to create a complete system model ``.
 - This model shows all the different components in a system, their interfaces, and their connections ``.

9. Architectural Design Decisions

- **Creative Process:** Architectural design is inherently a creative process, meaning its specific steps can vary depending on the type of system being developed ``.
- **Common Decisions:** Despite variations, certain common decisions are made across all design processes . These decisions significantly influence the non-functional characteristics of the system, such as performance, security, and maintainability.

9.1 Architecture Reuse

- **Domain Similarities:** Systems within the same application domain often exhibit similar architectures that reflect common domain concepts ``.
- **Application Product Lines:** Application product lines are typically constructed around a core architecture, with variations developed to satisfy specific customer requirements ``.
- **Architectural Patterns/Styles:** The architecture of a system can be designed around one or more architectural patterns or 'styles' ``.
 - These patterns encapsulate the essential nature of an architecture and can be instantiated (applied) in diverse ways ``.

9.2 Architecture and System Characteristics

Architectural decisions directly impact a system's non-functional characteristics:

- **Performance:** To enhance performance, critical operations should be localized, and communications minimized . It is also often beneficial to use large-grain components rather than fine-grain ones.
- **Security:** For security, a layered architecture is recommended, with critical assets placed in the innermost layers ``.
- **Safety:** To ensure safety, safety-critical features should be localized within a small number of sub-systems ``.
- **Availability:** To improve availability, redundant components and mechanisms for fault tolerance should be included in the design ``.
- **Maintainability:** (Specific architectural decisions for maintainability are not detailed in this section but are implied as a characteristic affected by architecture) ``.

10. Architectural Views

- **Purpose:** Architectural views address the question of which views or perspectives are most useful when designing and documenting a system's architecture ``.
- **Notations:** They also consider what notations should be employed for describing architectural models ``.
- **Multiple Views:** Each architectural model typically presents only one view or perspective of the system . For example, a model might show module decomposition, run-time process interaction, or the distribution of components across a network. For both effective design and thorough documentation, it is usually necessary to present multiple views of the software architecture ``.

10.1 4 + 1 View Model of Software Architecture

This model proposes several distinct views to describe a system's architecture:

- **Logical View:** This view illustrates the key abstractions within the system, typically represented as objects or object classes ``.
- **Process View:** This view shows how the system is composed of interacting processes at run-time ``.
- **Development View:** This view depicts how the software is decomposed for development purposes ``.
- **Physical View:** This view shows the system hardware and how software components are distributed across the processors in the system ``.
- **Use Cases/Scenarios (+1):** These views are related using use cases or scenarios, which provide the 'plus one' aspect of the model, showing how the different views interact ``.

10.2 Representing Architectural Views

- **UML (Unified Modeling Language):** Some argue that UML is an appropriate notation for describing and documenting system architectures ``.
- **Critique of UML:** However, a contrasting view suggests that UML does not include abstractions suitable for high-level system description ``.
- **ADLs (Architectural Description Languages):** These languages have been developed specifically for architectural description, but they are not yet widely used ``.

11. Architectural Patterns

- **Definition:** Patterns serve as a means of representing, sharing, and reusing knowledge . An architectural pattern is a stylized description of good design practice that has been tried and tested successfully in various environments.
- **Information Included:** Patterns should provide information about when they are useful and, equally important, when they are not suitable ``.
- **Representation:** Patterns can be represented using both tabular and graphical descriptions ``.

11.1 The Model-View-Controller (MVC) Pattern

- **Name:** MVC (Model-View-Controller) ``
- **Description:** This pattern separates presentation and user interaction from the underlying system data ``. The system is structured into three logical components that interact with each other:
 - **Model:** Manages the system data and associated operations on that data ``.
 - **View:** Defines and manages how the data is presented to the user ``.
 - **Controller:** Manages user interaction (e.g., key presses, mouse clicks) and passes these interactions to the View and the Model . (See Figure 6.3 in the original source for a visual representation).
- **Example:** Figure 6.4 illustrates the architecture of a web-based application system organized using the MVC pattern ``.
- **When Used:**
 - When there are multiple ways to view and interact with data ``.
 - When future requirements for interaction and presentation of data are unknown ``.
- **Advantages:**
 - Allows the data to change independently of its representation, and vice versa ``.

- Supports the presentation of the same data in different ways, with changes made in one representation automatically shown in all others ``.
- **Disadvantages:**
 - Can introduce additional code and increase code complexity, especially when the data model and interactions are simple ``.
- **Organization:** The Model-View-Controller pattern involves specific organizational relationships ``.
- **Application Example:** It is commonly used in web application architectures ``.

11.2 Layered Architecture

- **Name:** Layered architecture ``
- **Description:** This pattern is used to model the interfacing of sub-systems . It organizes the system into a set of layers, also known as abstract machines, where each layer provides a specific set of services to the layer above it. The lowest-level layers typically represent core services used throughout the system ``.
- **Support:** It supports the incremental development of sub-systems within different layers . When an interface in one layer changes, only the adjacent layer is affected, limiting the impact.
- **Example:** Figure 6.7 shows a layered model of a system for sharing copyright documents held in different libraries . (See Figure 6.6 for a generic representation of layered architecture).
- **When Used:**
 - When building new functionalities on top of existing systems ``.
 - When development is distributed across several teams, with each team responsible for a particular layer of functionality ``.
 - When there is a requirement for multi-level security ``.
- **Advantages:**
 - Allows for the replacement of entire layers, provided that the interface between layers is maintained ``.
 - Redundant facilities (e.g., authentication) can be provided in each layer, which increases the dependability of the system ``.
- **Disadvantages:**
 - Achieving a clean separation between layers can often be challenging in practice . A high-level layer may occasionally need to interact directly with lower-level layers, bypassing the immediate layer below it.
 - Performance can become an issue due to multiple levels of interpretation of a service request as it is processed at each layer ``.

- **Application Example:** The architecture of the iLearn system is an example of a layered architecture ``.

11.3 Repository Architecture

- **Name:** Repository ``
- **Description:** Sub-systems in a system often need to exchange data ``. This can be done in two primary ways:
 - Shared data is centrally managed in a database or repository, accessible by all sub-systems ``.
 - Each sub-system maintains its own database and explicitly passes data to other sub-systems ``.
 - The repository pattern specifically describes the former: All data in a system is managed in a central repository, and components interact only through this repository, not directly with each other ``.
- **Efficiency:** When large volumes of data need to be shared, the repository model is typically the most commonly used and efficient data-sharing mechanism ``.
- **Example:** Figure 6.9 provides an example of an Integrated Development Environment (IDE) where components utilize a repository of system design information . Each software tool generates information that becomes available for other tools to use. (See Figure 6.9 for a visual of a repository architecture for an IDE) ``.
- **When Used:**
 - When a system generates large volumes of information that must be stored for a long duration ``.
 - In data-driven systems where the inclusion of data in the repository triggers specific actions or tools ``.
- **Advantages:**
 - **Component Independence:** Components can operate independently as they do not need to be aware of the existence of other components ``.
 - **Propagation of Changes:** Changes made by one component can be automatically propagated to all other components that rely on that data ``.
 - **Consistent Data Management:** All data can be managed consistently (e.g., backups performed simultaneously) because it is centralized in one location ``.
- **Disadvantages:**
 - **Single Point of Failure:** The repository itself is a single point of failure, meaning problems within it can affect the entire system ``.
 - **Communication Inefficiencies:** Organizing all communication through the repository may introduce inefficiencies ``.

- **Distribution Difficulty:** Distributing the repository across several computers can be challenging ``.

11.4 Client-Server Architecture

- **Name:** Client-server ``
- **Description:** This is a distributed system model that illustrates how data and processing are distributed across various components . Although typically associated with distributed systems, it can also be implemented on a single computer. The architecture consists of:
 - A set of stand-alone servers that provide specific services (e.g., printing, data management) ``.
 - A set of clients that call upon and utilize these services ``.
 - A network that enables clients to access the servers ``.
 - In essence, the system's functionality is organized into services, with each service delivered from a separate server, and clients act as users of these services ``.
- **Example:** Figure 6.11 presents an example of a film and video/DVD library organized as a client-server system . (See Figure 6.11 for a visual of a client-server architecture for a film library).
- **When Used:**
 - When data in a shared database needs to be accessed from a range of different locations ``.
 - Can also be used when the load on a system is variable, as servers can be replicated to handle fluctuating demand ``.
- **Advantages:**
 - **Distribution:** The primary advantage is that servers can be distributed across a network, enabling remote access ``.
 - **Service Availability:** General functionality (e.g., a printing service) can be made available to all clients without needing to be implemented by each client individually ``.
- **Disadvantages:**
 - **Single Point of Failure (for services):** Each individual service acts as a single point of failure, making the system susceptible to denial-of-service attacks or server failures ``.
 - **Unpredictable Performance:** Performance can be unpredictable because it depends on the network's performance as well as the server's ``.
 - **Management Issues:** Management problems may arise if servers are owned by different organizations ``.

11.5 Pipe and Filter Architecture

- **Name:** Pipe and filter ``
- **Description:** In this pattern, functional transformations process their inputs to produce outputs . The processing of data in a system is organized so that each processing component (referred to as a 'filter') is discrete and performs a single type of data transformation. Data flows (like in a 'pipe') from one component to another for sequential processing . This model is analogous to UNIX shells.
- **Variants:** Variants of this approach are very common . When transformations occur sequentially, it is known as a batch sequential model, widely used in data processing systems.
- **Suitability:** This pattern is generally not suitable for interactive systems ``.
- **Example:** Figure 6.13 provides an example of a pipe and filter system used for processing invoices . (See Figure 6.13 for an example of pipe and filter architecture used in a payments system).
- **When Used:**
 - Commonly used in data processing applications (both batch- and transaction-based) where inputs are processed in separate stages to generate related outputs ``.
- **Advantages:**
 - **Clarity and Reuse:** Easy to understand and supports the reuse of individual transformations ``.
 - **Business Process Alignment:** The workflow style aligns well with the structure of many business processes ``.
 - **Evolution:** Evolution by adding new transformations is straightforward ``.
 - **Implementation Flexibility:** Can be implemented as either a sequential or a concurrent system ``.
- **Disadvantages:**
 - **Data Transfer Format Agreement:** A standardized format for data transfer must be agreed upon between communicating transformations ``.
 - **Overhead:** Each transformation component must parse its input and then unparse its output into the agreed-upon format . This increases system overhead.
 - **Reuse Limitations:** This overhead and format requirement may prevent the reuse of functional transformations that utilize incompatible internal data structures ``.

12. Application Architectures

- **Purpose:** Application systems are designed to fulfill specific organizational needs . Since many businesses share commonalities, their

application systems often possess a common architecture that reflects these shared application requirements.

- **Generic Application Architecture:** This is an architecture designed for a specific type of software system . It can be configured and adapted to create a system that meets particular requirements for an organization.

12.1 Use of Application Architectures

Generic application architectures are valuable for:

- As a starting point for architectural design ``.
- As a design checklist to ensure all aspects are considered ``.
- As a way of organizing the work of the development team ``.
- As a means of assessing components for potential reuse ``.
- As a common vocabulary for discussing and categorizing application types ``.

12.2 Examples of Application Types

- **Data Processing Applications:** These are data-driven applications that process data in batches without requiring explicit user intervention during the processing ``.
- **Transaction Processing Applications:** These are data-centered applications that process user requests and update information stored in a system database ``.
- **Event Processing Systems:** These are applications where system actions are dependent on interpreting events originating from the system's environment ``.
- **Language Processing Systems:** These are applications where user intentions are specified in a formal language, which is then processed and interpreted by the system ``.
 - **Specific Examples of Application Types:** Two widely used generic application architectures are transaction processing systems and language processing systems ``.

12.3 Transaction Processing Systems

- **Function:** These systems process user requests for information from a database or requests to update the database ``.
- **User Perspective of a Transaction:** From a user's perspective, a transaction is any coherent sequence of operations that achieves a specific goal ``.
 - **Example:** "Find the times of flights from London to Paris" is an example of a user transaction ``.

- **Processing:** Users make asynchronous requests for services, which are then processed by a transaction manager ``.
- **Structure:** The structure of transaction processing applications involves specific architectural elements (implied by Figure 6.14 in the original source) ``.
- **Examples:**
 - E-commerce systems ``
 - Reservation systems ``
 - The software architecture of an ATM system is a common example ``.

12.4 Information Systems Architecture

- **Generic Architecture:** Information systems possess a generic architecture that is often organized as a layered architecture ``.
- **Transaction-Based:** These systems are typically transaction-based, as interaction with them generally involves database transactions ``.
- **Common Layers:** The layers in such an architecture usually include:
 - The user interface ``
 - User communications ``
 - Information retrieval ``
 - The system database ``
- **Visual Representation:** Figure 6.15 illustrates a layered information system architecture, and the architecture of the Mentcare system is another example ``.

12.5 Web-Based Information Systems

- **Prevalence:** Information and resource management systems are now predominantly web-based, with user interfaces implemented using web browsers ``.
- **E-commerce Example:** E-commerce systems are Internet-based resource management systems that accept electronic orders for goods or services and then arrange for their delivery to the customer ``.
- **Application-Specific Layer:** In an e-commerce system, the application-specific layer often includes additional functionality, such as a "shopping cart" . This allows users to place multiple items in separate transactions and then pay for them all together.

12.5.1 Server Implementation for Web-Based Systems

- These systems are frequently implemented as multi-tier client-server architectures ``.
- **Components:**

- **Web Server:** Responsible for all user communications, with the user interface typically implemented via a web browser ``.
- **Application Server:** Responsible for implementing application-specific logic, as well as handling information storage and retrieval requests ``.
- **Database Server:** Manages the movement of information to and from the database and handles transaction management ``.

12.6 Language Processing Systems

- **Function:** These systems accept a natural or artificial language as input and generate some other representation of that language ``.
- **Interpreter:** They may also include an interpreter that acts upon the instructions specified in the processed language ``.
- **Use Cases:** Used when the easiest way to solve a problem is to describe an algorithm or define system data using a formal language ``.
 - **Example:** Meta-case tools process tool descriptions, method rules, etc., and then generate tools ``.
- **Architecture:** The architecture of a language processing system typically involves several stages of transformation ``.
- **Examples:**
 - Compilers ``
 - Command interpreters ``

12.6.1 Compiler Components

A typical compiler, as a language processing system, includes:

- **Lexical Analyzer:** Takes input language tokens and converts them into an internal form ``.
- **Symbol Table:** Holds information about the names of entities (e.g., variables, class names, object names) used in the text being translated ``.
- **Syntax Analyzer:** Checks the syntax (grammatical correctness) of the language being translated ``.
- **Syntax Tree:** An internal data structure that represents the program being compiled ``.
- **Semantic Analyzer:** Uses information from both the syntax tree and the symbol table to check the semantic correctness (meaningfulness) of the input language text ``.
- **Code Generator:** 'Walks' or traverses the syntax tree and generates abstract machine code ``.
- **Architectural Pattern Applications:** Language processing systems can be built using various architectural patterns, such as a repository

architecture (Figure 6.16) or a pipe and filter compiler architecture (Figure 6.17) ``.

13. Key Points Summary

- A software architecture is essentially a description of how a software system is organized ``.
- Architectural design decisions involve choices regarding the type of application, the system's distribution, and the architectural styles (patterns) to be used ``.
- Architectures can be documented from multiple perspectives or views, such as a conceptual view, a logical view, a process view, and a development view ``.
- Architectural patterns are a valuable means of reusing knowledge about generic system architectures . They describe the architecture and explain its applicability.
- Models of application system architectures help in understanding and comparing applications, validating application system designs, and assessing large-scale components for reuse ``.
- Transaction processing systems are interactive systems that enable multiple users to remotely access and modify information within a database ``.
- Language processing systems are used to translate texts from one language into another and to execute instructions specified in the input language . They typically comprise a translator and an abstract machine.

UNIT-II: System Modeling, Architectural Design, Design and Implementation

Chapter 7 – Design and Implementation

1. Design and Implementation

Software design and implementation constitute the stage within the software engineering process where an executable software system is developed. These activities are invariably inter-leaved.

- **Software Design:** This is a creative activity focused on identifying software components and their relationships, based on a customer's requirements.
- **Implementation:** This is the process of realizing the design as a program.

1.1. Build or Buy

In many domains, it is now possible to acquire off-the-shelf systems, often referred to as COTS (Commercial Off-The-Shelf) systems, which can be adapted and tailored to meet user requirements. For instance, to implement a medical records system, one can purchase an existing package already in use in hospitals. This approach can be cheaper and faster than developing a system using a conventional programming language.

When an application is developed in this manner, the design process shifts its focus to how the system's configuration features can be utilized to deliver the specified system requirements.

2. Object-Oriented Design Using the UML

2.1. An Object-Oriented Design Process

Structured object-oriented design processes involve the development of various system models. While these processes demand significant effort for the development and maintenance of these models, which might not be cost-effective for small systems, they are crucial as an important communication mechanism for large systems developed by different groups.

2.1.1. Process Stages

There is a variety of different object-oriented design processes, which depend on the organization utilizing the process. However, common activities found in these processes typically include:

- Defining the context and modes of use of the system.
- Designing the system architecture.
- Identifying the principal system objects.
- Developing design models.
- Specifying object interfaces.

This process can be illustrated using a design for a wilderness weather station.

2.2. System Context and Interactions

Understanding the relationships between the software being designed and its external environment is essential. This understanding helps in deciding how to provide the required system functionality and how to structure the system to communicate with its environment. Furthermore, comprehending the context allows for the establishment of system boundaries, which aids in determining which features will be implemented within the designed system and which will reside in other associated systems.

2.2.1. Context and Interaction Models

- **System Context Model:** This is a structural model that demonstrates other systems present in the environment of the system under development.
- **Interaction Model:** This is a dynamic model that illustrates how the system interacts with its environment during its use.

2.2.2. Example: System Context for the Weather Station

The weather station system's context involves interactions with external entities.

2.2.3. Example: Weather Station Use Cases

A use case describes how an external actor interacts with the system to achieve a specific goal.

- **Use Case Description—Report Weather**
 - **System:** Weather station
 - **Use case:** Report weather
 - **Actors:** Weather information system, Weather station
 - **Description:** The weather station transmits a summary of collected weather data from its instruments during a collection period to the weather information system. The data includes maximum, minimum, and average ground and air temperatures; maximum, minimum, and average air pressures; maximum, minimum, and average wind speeds; total rainfall; and wind direction, sampled at five-minute intervals.
 - **Stimulus:** The weather information system establishes a satellite communication link with the weather station and requests data transmission.
 - **Response:** The summarized data is sent to the weather information system.
 - **Comments:** Weather stations are typically requested to report once per hour, though this frequency may vary between stations and could be modified in the future.

3. Architectural Design

Once the interactions between the system and its environment are understood, this information is used for designing the system architecture. This involves identifying the major components that constitute the system and their interactions. These components may then be organized using an architectural pattern, such as a layered or client-server model.

For example, the weather station system is composed of independent subsystems that communicate by broadcasting messages on a common infrastructure.

4. Object Class Identification

Identifying object classes is frequently a challenging aspect of object-oriented design. There is no "magic formula" for object identification; it relies heavily on the skill, experience, and domain knowledge of the system designers. Object identification is an iterative process, meaning it is unlikely to be perfected on the first attempt.

4.1. Approaches to Identification

Several approaches can be used for object identification:

- Employing a grammatical approach based on a natural language description of the system.
- Basing the identification on tangible entities found in the application domain.
- Utilizing a behavioral approach, where objects are identified based on their participation in specific behaviors.
- Performing a scenario-based analysis, where objects, attributes, and methods within each scenario are identified.

4.2. Example: Weather Station Object Classes

Object class identification in the weather station system can be based on the tangible hardware and data within the system:

- **Ground thermometer, Anemometer, Barometer:** These are application domain objects that are 'hardware' objects, related to the instruments in the system.
- **Weather station:** This represents the basic interface of the weather station to its environment, thus reflecting the interactions identified in the use-case model.
- **Weather data:** This encapsulates the summarized data collected from the instruments.

5. Design Models

Design models illustrate the objects, object classes, and the relationships between these entities. There are two primary kinds of design models:

- **Structural Models:** These describe the static structure of the system in terms of object classes and their relationships.
- **Dynamic Models:** These describe the dynamic interactions that occur between objects.

5.1. Examples of Design Models

Examples of specific design models include:

- **Subsystem Models:** These models show logical groupings of objects into coherent subsystems.
- **Sequence Models:** These models depict the sequence of interactions between objects.
- **State Machine Models:** These models show how individual objects change their state in response to events.
- **Other Models:** This category includes use-case models, aggregation models, generalization models, and more.

5.1.1. Subsystem Models

Subsystem models demonstrate how the design is organized into logically related groups of objects. In the UML (Unified Modeling Language), these are typically represented using packages, which are encapsulation constructs. These models are logical, and the actual organization of objects within the system might differ.

5.1.2. Sequence Models

Sequence models illustrate the temporal sequence of object interactions.

- Objects are arranged horizontally across the top of the diagram.
- Time is represented vertically, meaning models are read from top to bottom.
- Interactions are shown as labeled arrows, with different arrow styles indicating different types of interaction.
- A thin rectangle on an object's lifeline signifies the period during which that object is the controlling object in the system.

Example: Sequence Diagram Describing Data Collection (Note: The specific diagram content is not provided, but its purpose is to show the sequence of interactions for data collection.)

5.1.3. State Diagrams

State diagrams are utilized to illustrate how objects respond to various service requests and the state transitions that these requests trigger. They serve as useful high-level models for understanding a system's or an object's run-time behavior. It is generally not necessary to create a state diagram for every object in the system, as many objects are relatively simple, and a state model would introduce unnecessary detail to the design.

Example: Weather Station State Diagram (Note: The specific diagram content is not provided, but its purpose is to show the state transitions of the weather station.)

6. Interface Specification

Object interfaces must be precisely specified to enable parallel design of objects and other components. Designers should avoid designing the interface representation directly; instead, this detail should be hidden within the object itself. Objects may possess multiple interfaces, which serve as different viewpoints on the methods they provide. The UML employs class diagrams for interface specification, though Java can also be used for this purpose.

Example: Weather Station Interfaces (Note: Specific interface details are not provided, but the concept is applied to the weather station.)

7. Design Patterns

A design pattern is a method for reusing abstract knowledge pertaining to a problem and its corresponding solution. A pattern provides a description of the problem and the fundamental essence of its solution. It must be sufficiently abstract to allow for reuse in diverse settings. Pattern descriptions typically leverage object-oriented characteristics such as inheritance and polymorphism.

Patterns and Pattern Languages are ways to describe best practices, good designs, and to capture experience in a manner that facilitates its reuse by others.

7.1. Pattern Elements

Design patterns are typically described using several key elements:

- **Name:** A meaningful identifier for the pattern.
- **Problem Description:** A detailed explanation of the problem the pattern addresses.
- **Solution Description:** This is not a concrete design but rather a template for a design solution that can be instantiated in various ways.

- **Consequences:** The results and trade-offs that arise from applying the pattern.

7.2. The Observer Pattern

The Observer pattern is a widely recognized design pattern.

- **Name:** Observer.
- **Description:** The Observer pattern separates the display of an object's state from the object itself, allowing for alternative displays. When the object's state changes, all associated displays are automatically notified and updated to reflect the change.
- **Problem Description:** In many scenarios, it is necessary to provide multiple displays of state information, such as a graphical display and a tabular display. Not all of these display requirements may be known when the information is initially specified. All alternative presentations should support interaction, and all displays must be updated when the state changes. This pattern is applicable whenever more than one display format for state information is required, and the object maintaining the state information does not need to be aware of the specific display formats used.
- **Solution Description:** The Observer pattern involves two abstract objects: `Subject` and `Observer`, and two concrete objects: `ConcreteSubject` and `ConcreteObserver`. These concrete objects inherit attributes from their related abstract objects. The abstract objects include general operations applicable in all situations. The state to be displayed is maintained in `ConcreteSubject`. `ConcreteSubject` inherits operations from `Subject`, enabling it to add and remove `Observers` (each observer corresponds to a display) and to issue notifications when its state has changed. The `ConcreteObserver` maintains a copy of the state of `ConcreteSubject` and implements the `Update()` interface from `Observer`, which ensures these copies remain synchronized. The `ConcreteObserver` automatically displays the state and reflects changes whenever the state is updated.
- **Consequences:** The `Subject` object only knows about the abstract `Observer` and lacks details about the concrete observer classes. This results in minimal coupling between these objects. However, due to this lack of knowledge, optimizations that could enhance display performance become impractical. Changes to the subject may cause a set of (further details omitted but implies broader impact).

Example: Multiple Displays Using the Observer Pattern (Note: The specific example or diagram content is not provided, but the pattern's application is highlighted.)

Example: A UML Model of the Observer Pattern (Note: The specific diagram content is not provided, but its existence as an illustration of the pattern is mentioned.)

7.3. Design Problems and Patterns

To effectively use patterns in design, a designer needs to recognize when a specific design problem might have an associated pattern that can be applied. Examples of such problems and their relevant patterns include:

- **Notifying multiple objects about a state change:** The Observer pattern is used when several objects need to be informed that the state of another object has changed.
- **Tidying up interfaces of related objects:** The Façade pattern is applicable when there is a need to simplify the interfaces to a number of related objects that have often been developed incrementally.
- **Providing a standard way to access collection elements:** The Iterator pattern offers a consistent method for accessing elements within a collection, regardless of how that collection is implemented.
- **Extending functionality of an existing class at run-time:** The Decorator pattern allows for the possibility of extending the functionality of an existing class dynamically during runtime.

8. Implementation Issues

While programming is crucial, the focus here is on other implementation issues typically not covered in programming texts. These include:

- **Reuse:** Most modern software is built by reusing existing components or systems. When developing software, maximizing the use of existing code is important.
- **Configuration Management:** Throughout the development process, it is necessary to track the numerous different versions of each software component using a configuration management system.
- **Host-Target Development:** Production software usually executes on a separate machine (the target) from where it was developed (the host).

8.1. Reuse

From the 1960s to the 1990s, the majority of new software was developed from scratch, with all code written in a high-level programming language. The only significant software reuse during this period involved functions and objects found in programming language libraries. However, increasing costs and schedule pressures rendered this approach increasingly unviable, especially for commercial and Internet-based systems. Consequently, a development approach centered around the reuse of existing software emerged and is now commonly employed for business and scientific software.

8.1.1. Reuse Levels

Software reuse can occur at several distinct levels:

- **The Abstraction Level:** At this level, software itself is not directly reused. Instead, knowledge of successful abstractions is applied in the design of new software.
- **The Object Level:** Here, objects from a library are directly reused, eliminating the need to write the code from scratch.
- **The Component Level:** Components are collections of objects and object classes that are reused within application systems.
- **The System Level:** At the highest level, entire application systems are reused.

8.1.2. Reuse Costs

While reuse offers benefits, it also entails various costs:

- The time spent searching for reusable software and assessing whether it meets specific needs.
- The cost of purchasing reusable software, which can be very high for large off-the-shelf systems.
- The costs associated with adapting and configuring the reusable software components or systems to align with the requirements of the developing system.
- The costs of integrating reusable software elements with each other (if sourced from different origins) and with any newly written code.

8.2. Configuration Management

Configuration management refers to the general process of managing a changing software system. Its primary goal is to support the system integration process, ensuring that all developers can access project code and documents in a controlled manner, track changes, and compile and link components to construct

a system. (Further information can be found in Chapter 25, though not provided here).

8.2.1. Configuration Management Activities

Key activities within configuration management include:

- **Version Management:** This provides support for tracking different versions of software components. Version management systems also include features to coordinate development among multiple programmers.
- **System Integration:** This activity supports developers in defining which versions of components are used to create each version of a system. This definition is then used to automatically build a system by compiling and linking the necessary components.
- **Problem Tracking:** This offers support for users to report bugs and other issues. It also allows all developers to see who is addressing these problems and when they are resolved.

8.3. Host-Target Development

Most software development occurs on one computer, known as the **host**, but the software is intended to run on a separate machine, called the **target**. More broadly, this distinction can be discussed in terms of a **development platform** and an **execution platform**. A platform encompasses more than just hardware; it includes the installed operating system along with other supporting software, such as a database management system, or, for development platforms, an interactive development environment. The development platform typically has different installed software than the execution platform, and these platforms may even have different architectures.

8.3.1. Development Platform Tools

Development platforms commonly include a suite of tools to assist in the software development process:

- An integrated compiler and syntax-directed editing system that enables the creation, editing, and compilation of code.
- A language debugging system.
- Graphical editing tools, such as those used for editing UML models.
- Testing tools, like JUnit, which can automatically run a set of tests on a new version of a program.
- Project support tools that help organize code for various development projects.

8.3.2. Integrated Development Environments (IDEs)

Software development tools are frequently grouped together to form an Integrated Development Environment (IDE). An IDE is a collection of software tools that supports different facets of software development within a common framework and user interface. IDEs are often designed to support development in a specific programming language, such as Java. A language-specific IDE may be custom-developed or may be an instantiation of a general-purpose IDE with added language-specific support tools.

8.3.3. Component/System Deployment Factors

Several factors influence where components and systems are deployed:

- If a component is designed for a specific hardware architecture or depends on another software system, it must be deployed on a platform that provides the necessary hardware and software support.
- High availability systems might require components to be deployed on more than one platform. This ensures that an alternative implementation of the component is available in the event of a platform failure.
- If there is a significant amount of communication traffic between components, it is generally more efficient to deploy them on the same platform or on physically close platforms. This minimizes the delay between when a message is sent by one component and received by another.

9. Open Source Development

Open source development is a software development approach where the source code of a software system is published, and volunteers are invited to participate in the development process. Its origins lie with the Free Software Foundation (www.fsf.org), which advocates that source code should not be proprietary but should always be available for users to examine and modify as they wish. Open source software extended this concept by leveraging the Internet to recruit a much larger pool of volunteer developers, many of whom are also users of the software.

9.1. Open Source Systems

The most widely known open source product is the Linux operating system, which is extensively used as a server system and increasingly as a desktop environment. Other significant open source products include Java, the Apache web server, and the MySQL database management system.

9.2. Open Source Issues

When considering open source, two key issues often arise:

- Should the product being developed incorporate open source components?
- Should an open source approach be utilized for the software's development?

9.3. Open Source Business

An increasing number of product companies are adopting an open source approach to development. Their business model typically does not rely on selling a software product but rather on selling support for that product. These companies believe that involving the open source community allows software to be developed more cheaply, more quickly, and fosters a community of users for the software.

9.4. Open Source Licensing

A fundamental principle of open source development is that source code should be freely available. However, this does not imply that anyone can do whatever they wish with the code. Legally, the developer of the code (whether a company or an individual) retains ownership. They can impose restrictions on how the code is used by including legally binding conditions within an open source software license.

Some open source developers advocate that if an open source component is used to develop a new system, then that new system should also be open source. Others are willing to permit their code to be used without this restriction, allowing the developed systems to be proprietary and sold as closed source systems.

9.4.1. License Models

Several prominent open source license models exist:

- **The GNU General Public License (GPL):** This is a 'reciprocal' license, meaning that if you use open source software licensed under the GPL, you are required to also make your derivative software open source.
- **The GNU Lesser General Public License (LGPL):** This is a variant of the GPL. It allows developers to create components that link to open source code without being obliged to publish the source code of these new components.

- **The Berkeley Standard Distribution (BSD) License:** This is a non-reciprocal license. It means you are not compelled to re-publish any changes or modifications made to the open source code. You are permitted to include the code in proprietary systems that are sold.

9.5. License Management

Effective management of open source licenses involves several practices:

- Establishing a system for maintaining information about downloaded and used open source components.
- Being aware of the different types of licenses and understanding how a component is licensed before it is used.
- Being aware of the evolution pathways for components.
- Educating individuals about open source principles and practices.
- Implementing auditing systems to ensure compliance.
- Actively participating in the open source community.